



Project Presentation

Hulon Jenkins

Agenda

- About Me
- University/Microsoft Projects
- Personal Project Overview
 - Planning
 - Architecture
 - Parts Selection
 - Schematic & Layout
 - Roadblocks and Debugging
 - Software/Firmware Quick Look
 - Results and Lessons Learned
- How I Engineer
- Questions
- Demo (if time)

About Me – Hulon Jenkins

Focus

- Embedded hardware and electronics design and bring up

Education

- BSE Computer Engineering from University of Michigan
- Currently pursuing MS Electrical Engineering part time through University of Colorado Boulder

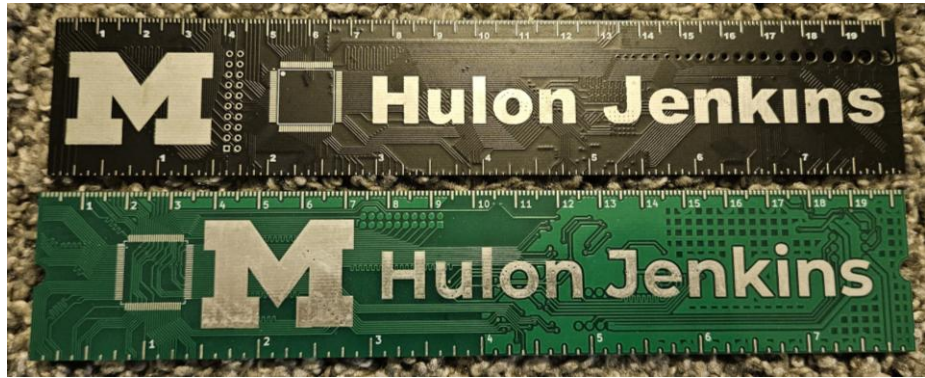
Experience

- Embedded Hardware Team Lead and member on Michigan Mars Rover Team
- Currently work at Microsoft (C++ Compiler)

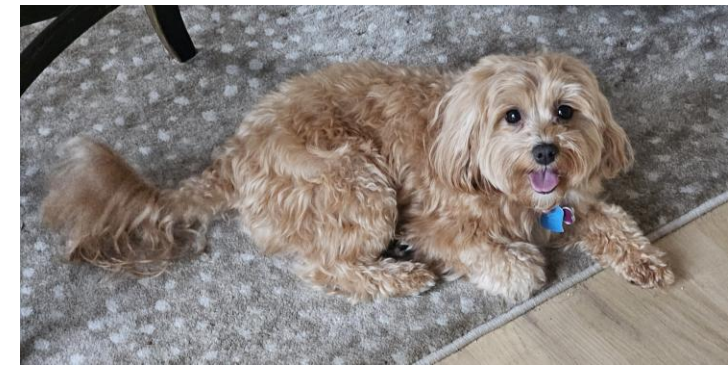
Interests

- Spaceflight, Table Tennis, 3d Printing

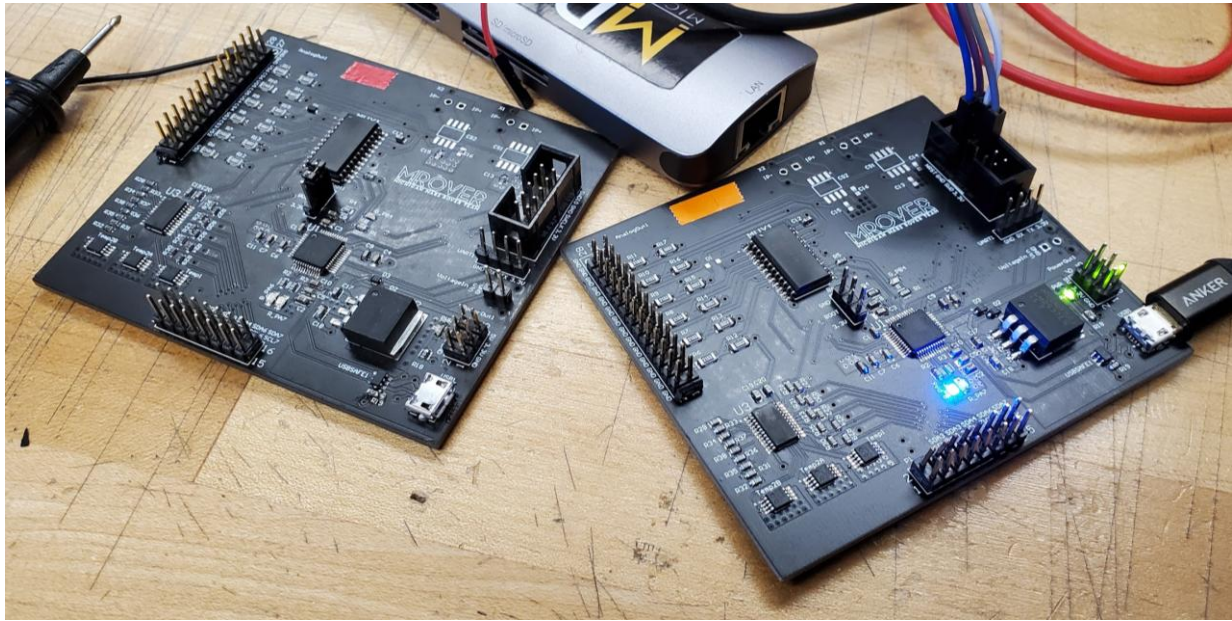




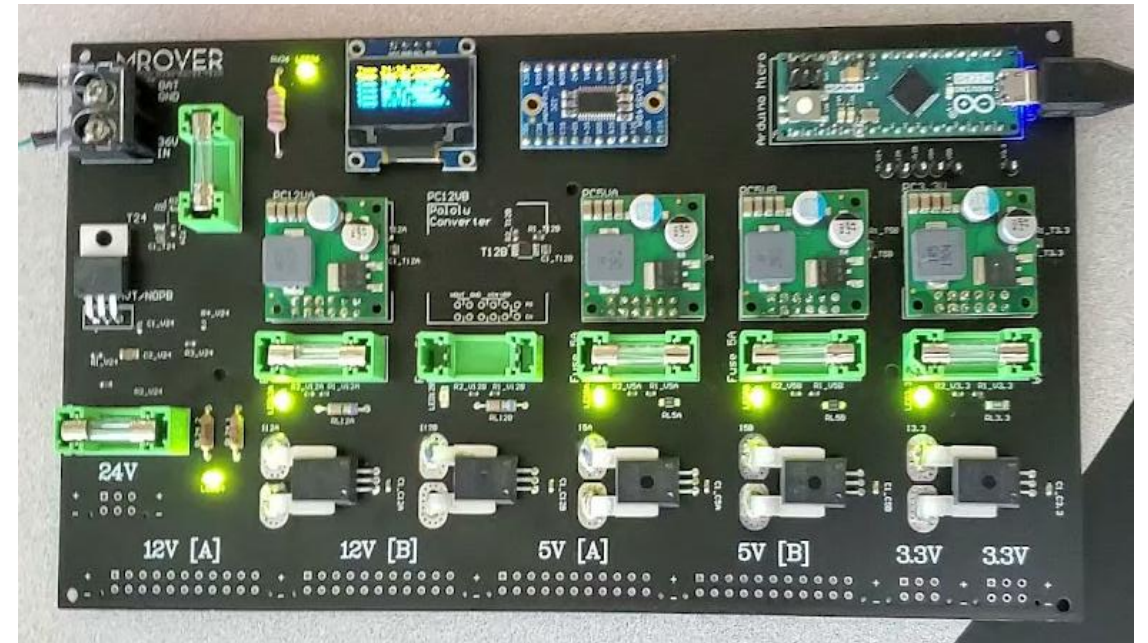
M!ROVER
MICHIGAN MARS ROVER TEAM



University - MROVER



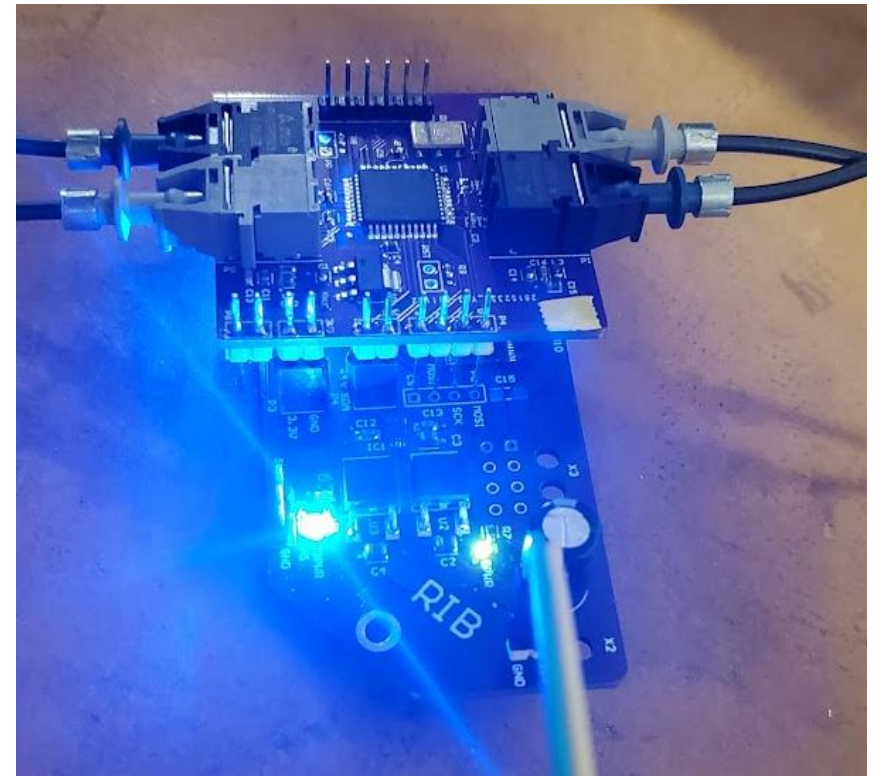
STM32 Analog and I2C Mux
Sensor Board. In support of
custom hardware for the Rover.



Power Distribution Board for
Rover.

University - Classes

- A Drop-In Wire Replacement using Fiber Optics
- Is Protocol Agnostic
- Replicates signals on wires. Which can be interpreted as I2C, SPI, UART, GPIO, etc.
- We used it to replicate I2C over a length of 5 meters

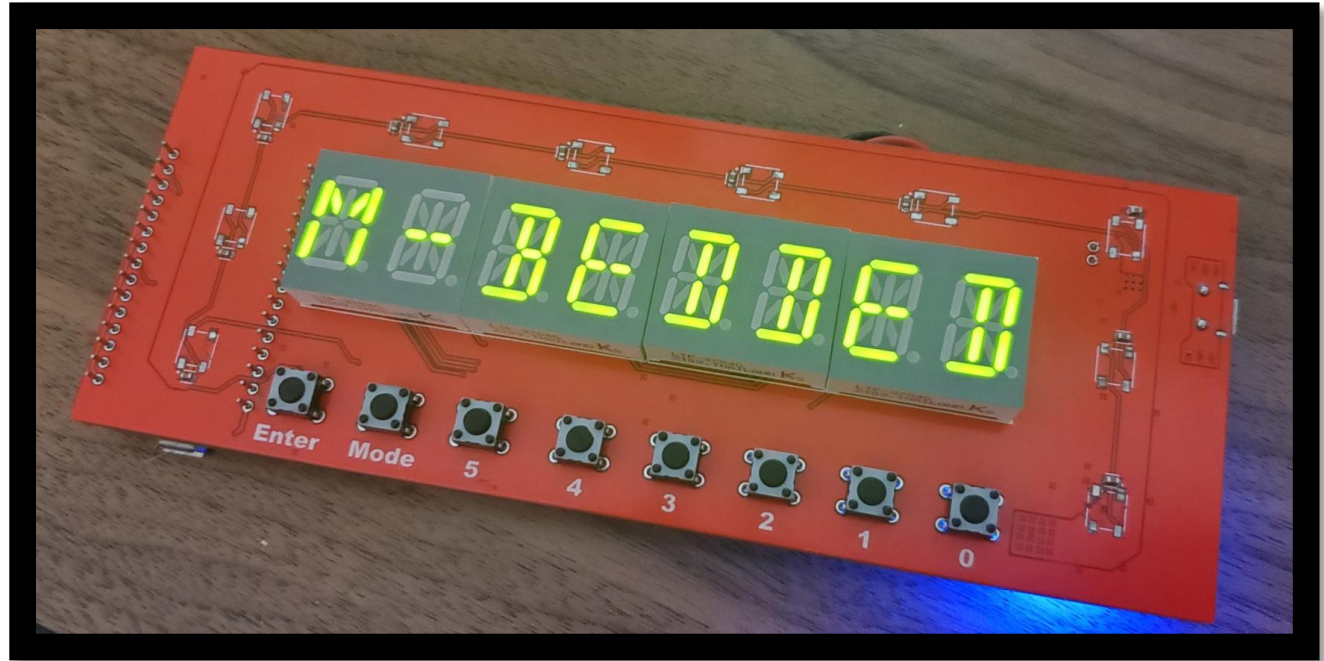


Microsoft

- Pattern Matching Software Security Tool Team
 - Implemented patterns and logic to statically match a secret and validate if that secret was still live
- C++ Compiler Team
 - Worked on a large optimization with tail calling which is used in upcoming Python Interpreter!

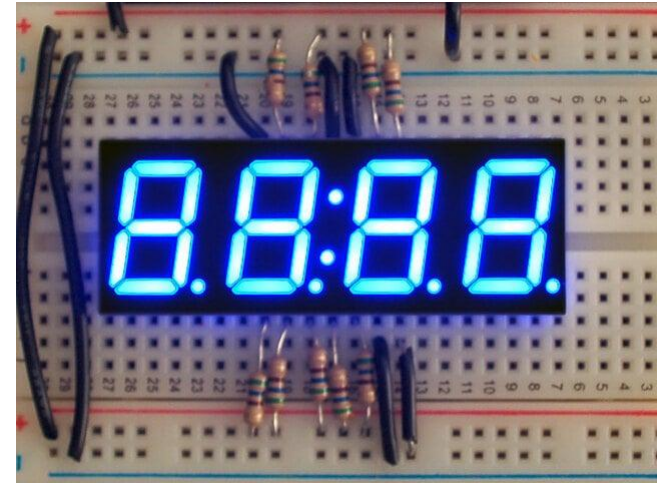
Personal Project Overview

- ESP8266 based 14-segment display board
- Battery powered
- 8 Button input
- Wifi Connected



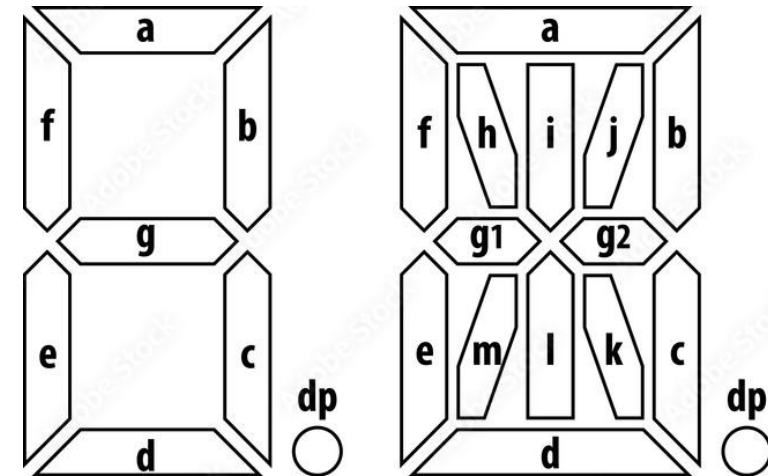
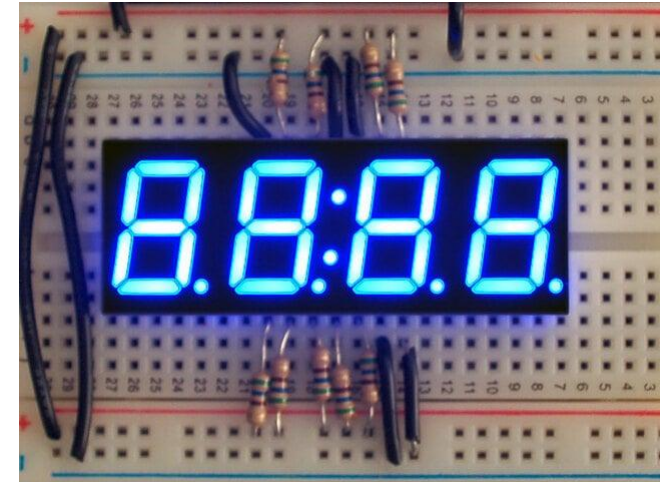
Project Requirements & Initial Planning

- Main goal, be able to display real time stock data on hex displays
 - Must be internet connected



Project Requirements & Initial Planning

- Main goal, be able to display real time stock data on hex displays
 - Must be internet connected
- 14-segment allows for alphanumeric characters.
- Can display temperature data as well
- Mobile, so battery powered
- All interfacing will be through the 14-segments displays
 - Would need buttons for inputs
- Have LEDs that can change color/brightness as well



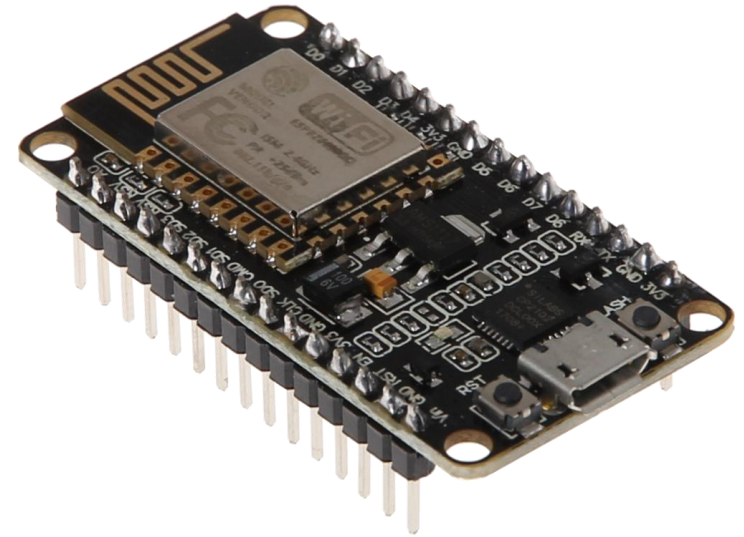
Design and Architecture (1/2)

Connectivity and Main Controller

- 3.3V Esp8266 for Wi-Fi connectivity

Power

- 3.7V rechargeable LiPo battery
- Passthrough charging IC (Can charge battery and use simultaneously)
- Buck Converter for 3.7V $\rightarrow$ 3.3V
- Boost Converter for 3.7V $\rightarrow$ 5V



Design and Architecture (2/2)

Outputs

- 3.3V 14-Segment Displays
- 5V individually addressable NeoPixel LEDs



Inputs

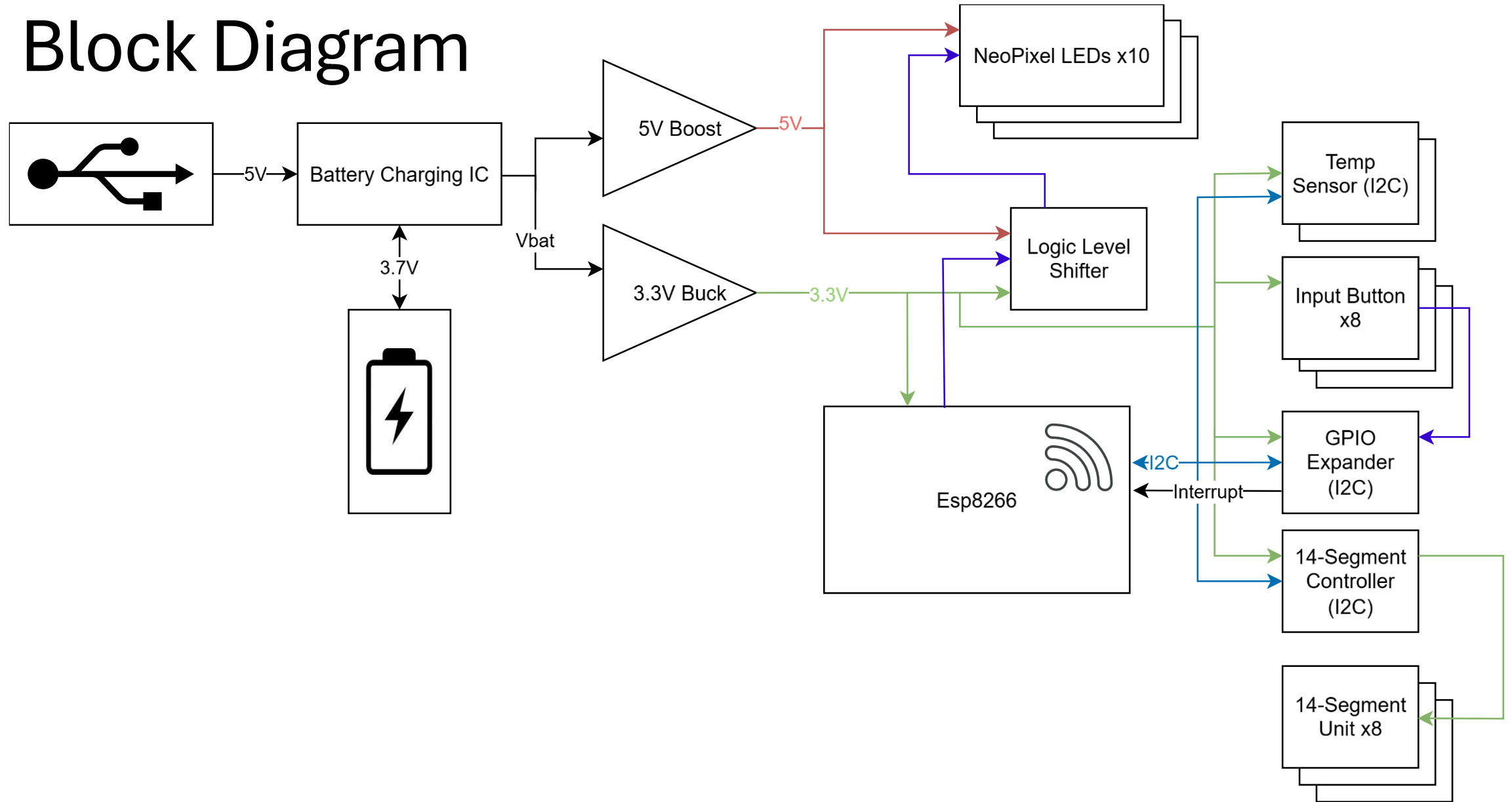
- Multiple tactile switches



Logic

- Logic level shifter for 3.3V Esp8266 logic to 5V for NeoPixels
- 14-Segment Control IC for simplicity
- 3.3V Temperature sensors

Block Diagram



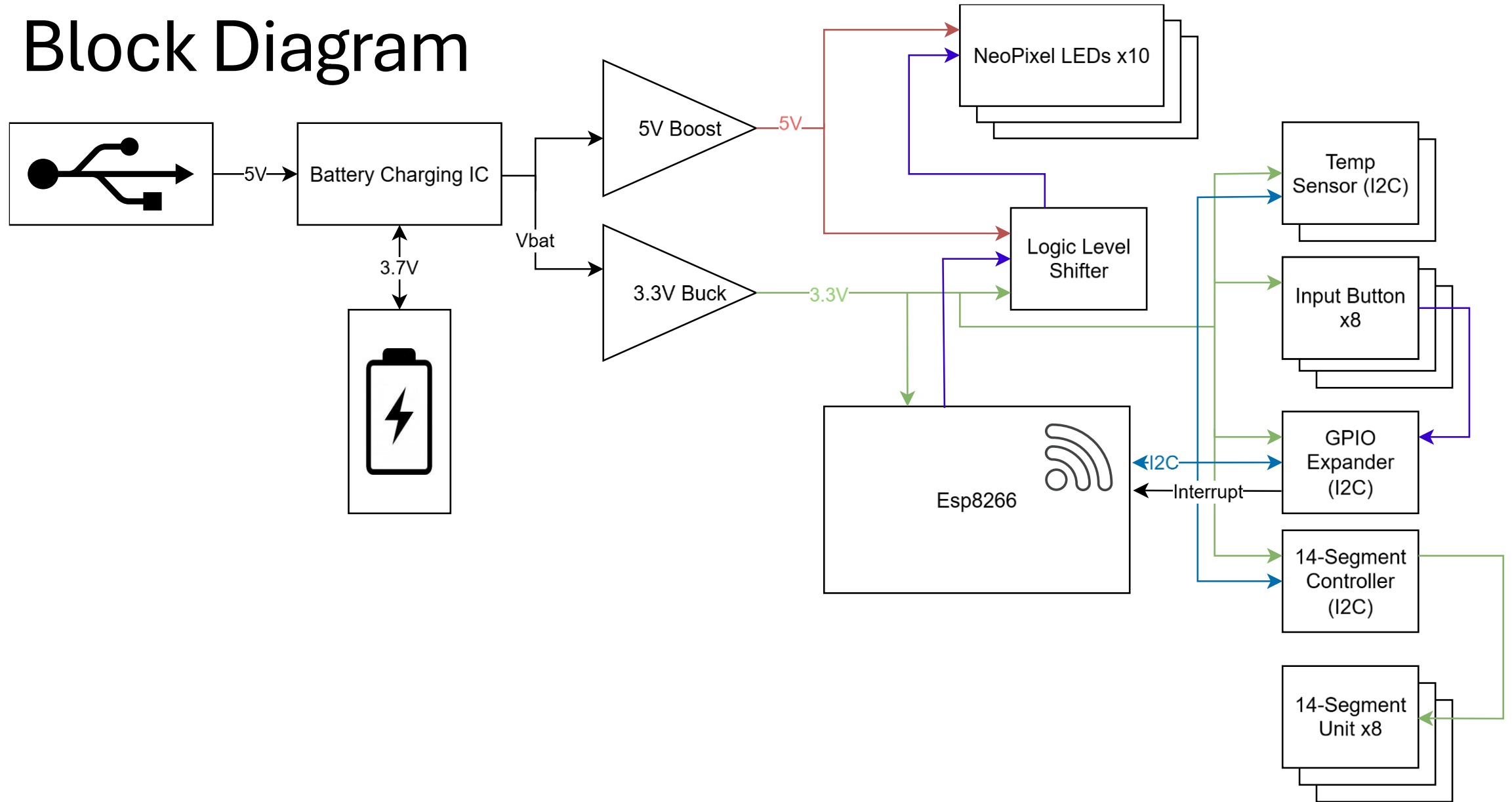
Implementation Details (1/2)

Functionality	IC Selected	Why
Battery Charger	MCP73871	CC/CV, Load Sharing + Battery Charging, Indication and Safety (temp, short protection)
14-Segment Controller	MAX6955	I2C, Can control 8 14-segments, can PWM LEDs to reduce current draw
Buck Converter	MAX17623	Up to 1A @ 3.3V
Boost Converter	TPS61090	Up to 500mA @ 5V
GPIO Expander	TCA9534PWR	I2C, 8 inputs, has interrupt pin
Logic Level Shifter	SN74LVC1T45-Q1	Converters 3.3V logic to 5V with 3.9ns delay
Temperature Sensors	MCP9808-E/MS	I2C, address pins, 0.0625C accuracy
NeoPixels	NeoPixel 1655	Multicolor, individually addressable

Implementation Details (2/2)

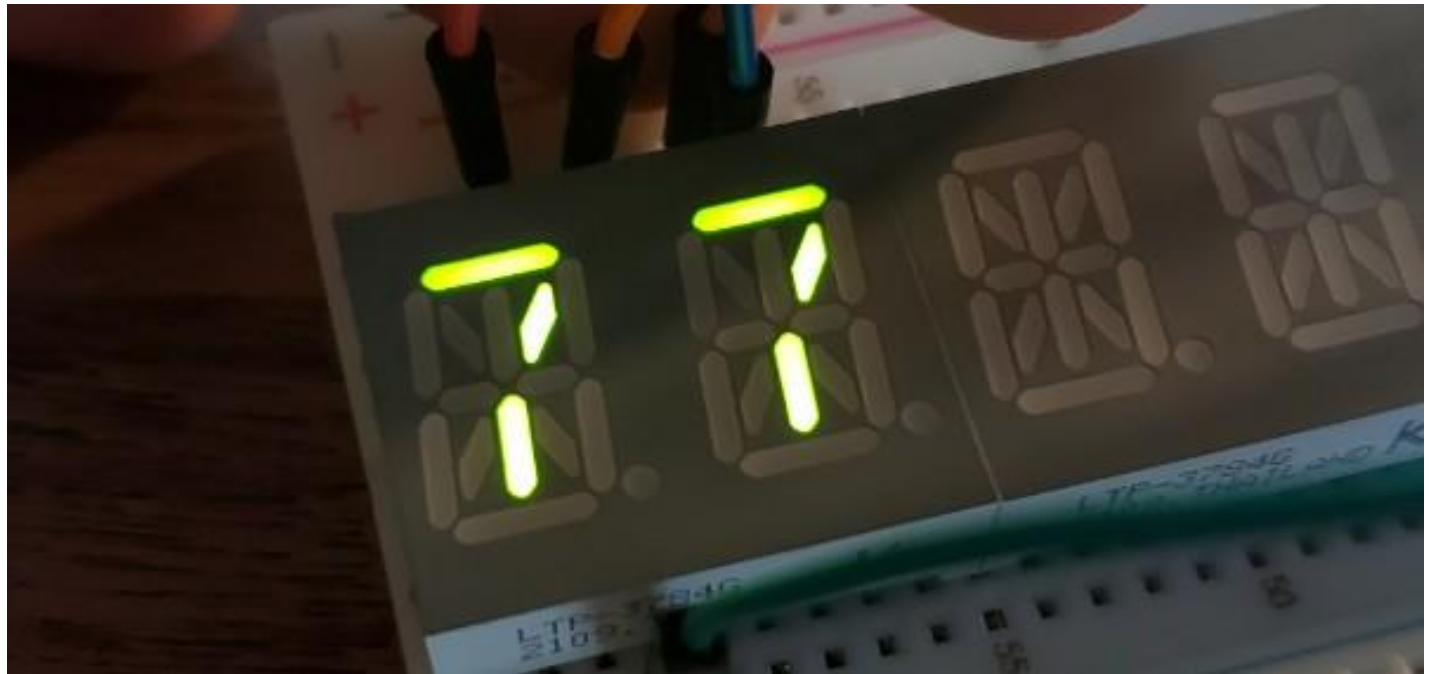
Functionality	Details	Why
MCU	ESP8266 Dev Board	I had the dev board in hand already, was going to have enough external circuitry to test
USB	Micro USB connector	The Esp8266 already has a Micro USB. I'd prefer USB Type C, but would prefer consistency
Buttons	Tactile Switches	Cheap easy to use with pull-down resistor
Battery	3000 mAh 3.7V Battery	Expected $\sim 40\text{mA} \cdot 8 + \sim 30\text{mA} \cdot 10 + \sim 100\text{mA} = 720\text{mA}$ of current, so would last a ~ 4 hours on battery power at full brightness

Block Diagram

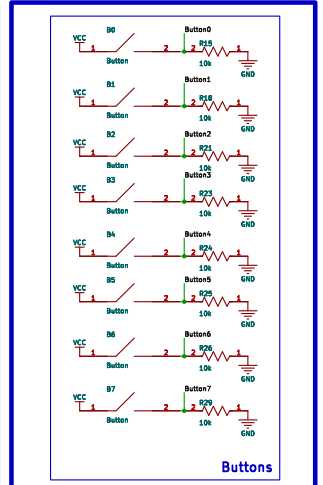
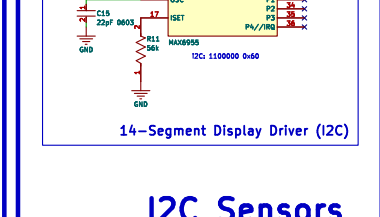
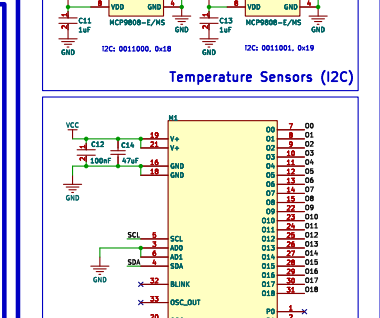
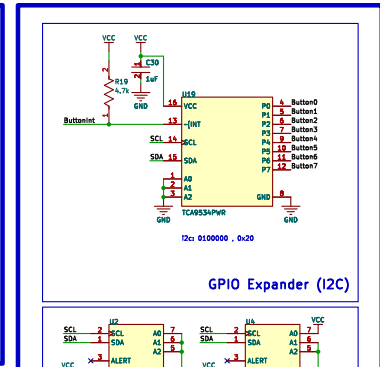
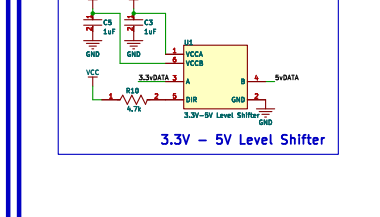
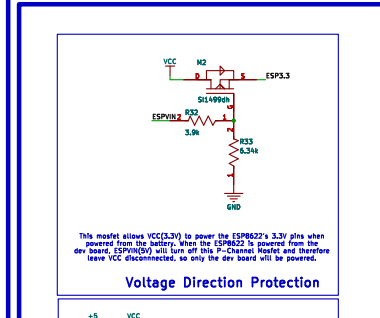
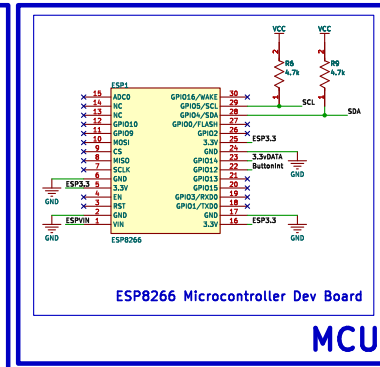
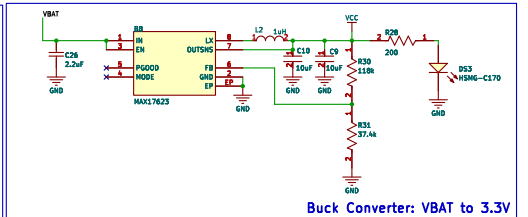
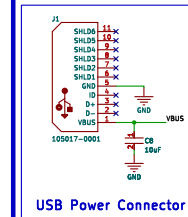
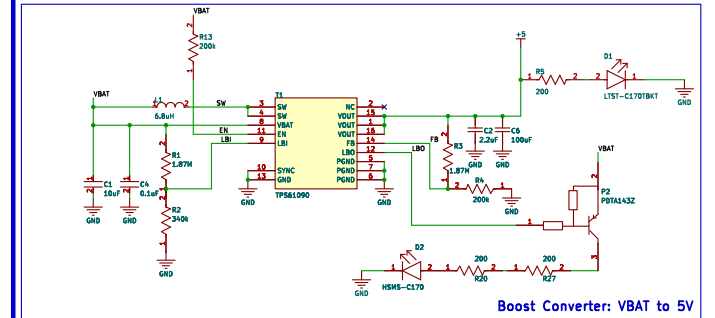
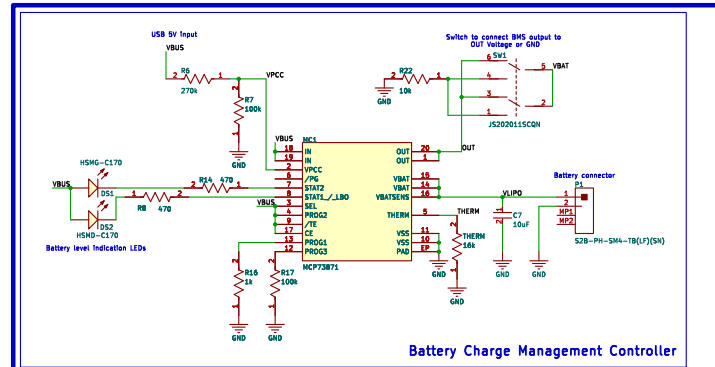


Initial Trials

- Breadboarded 14-segment displays and Esp8266 connectivity and control



Schematic

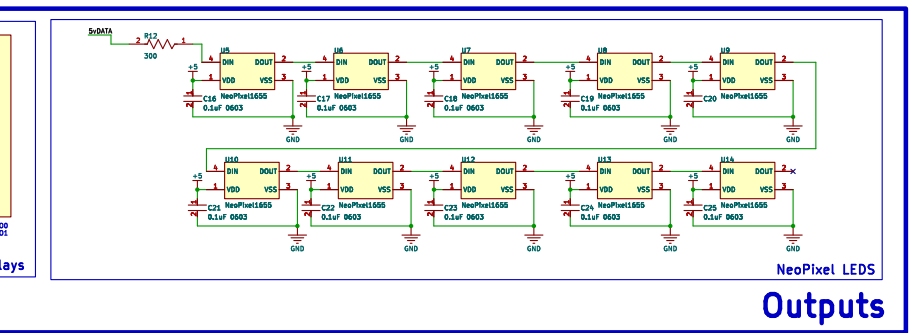
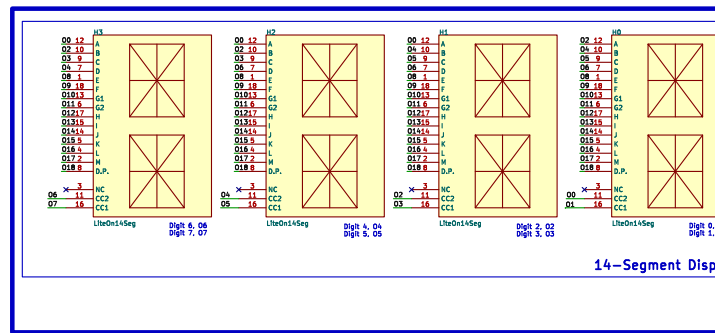


Inputs

Power

PCB Interfacing

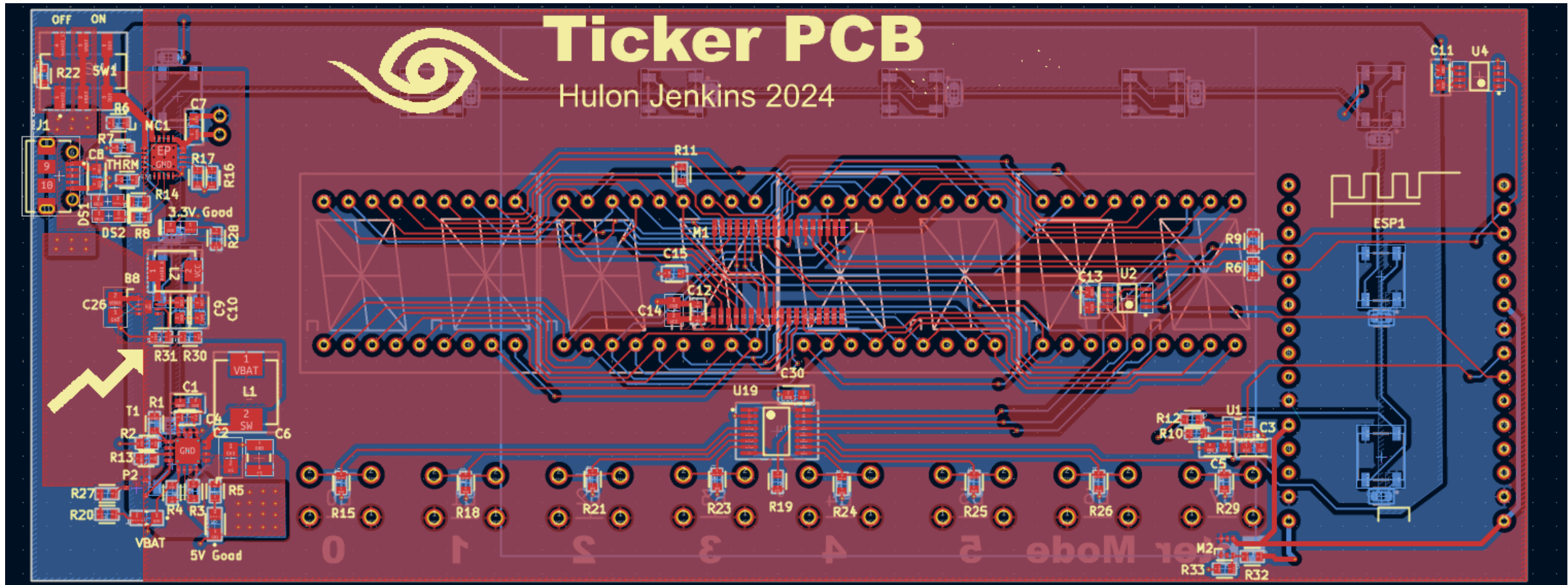
I2C Sensors & Controllers



Outputs

Note: I added voltage direction protection since I was mounting a dev board onto a PCB for this project

PCB layout (2 layer)

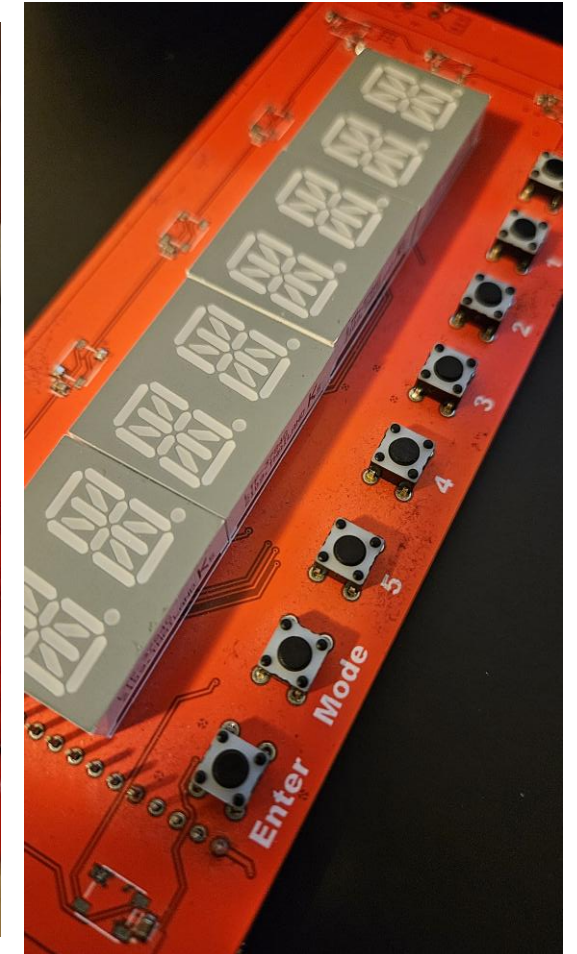
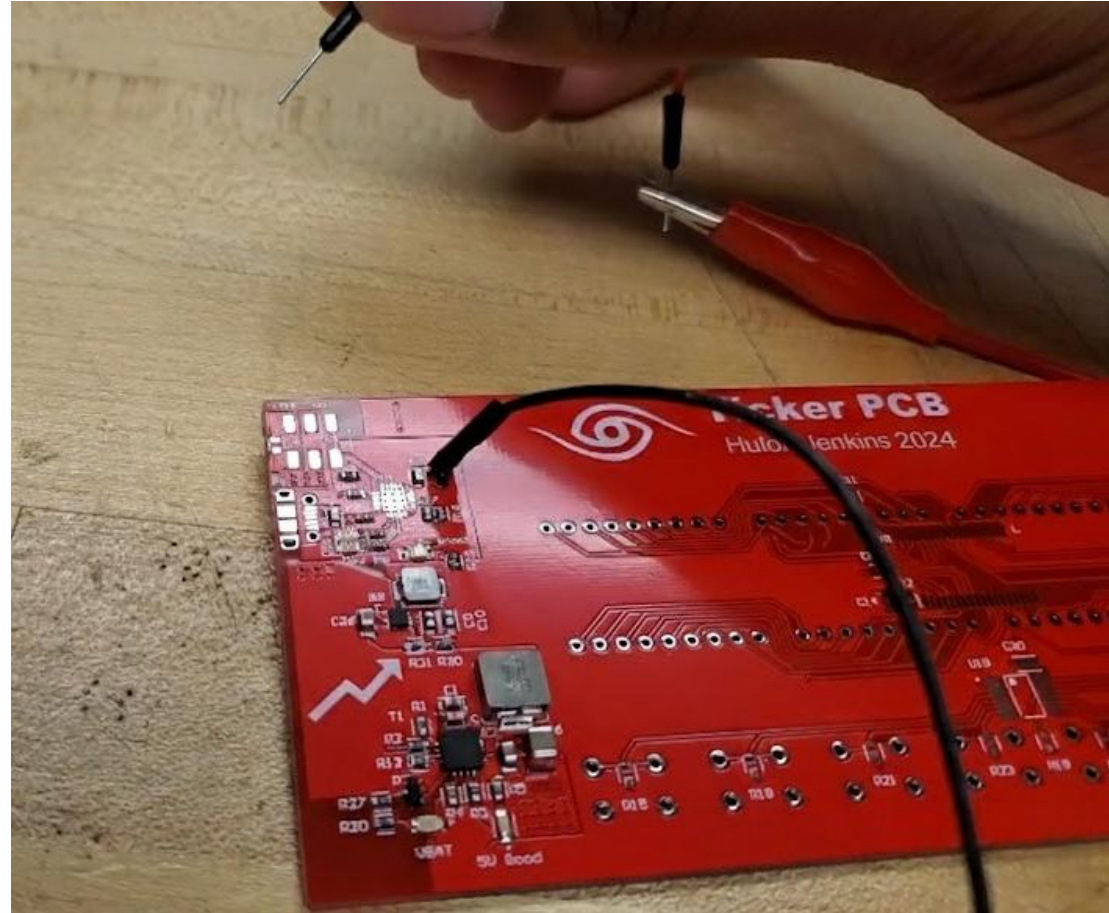


Integration and Build Process

I first soldered the buck and boost converters and battery charging IC.

I powered it with USB (no battery) and verified the 3.3V and 5V rails were good

I soldered on the ESP8266 and verified it powered on

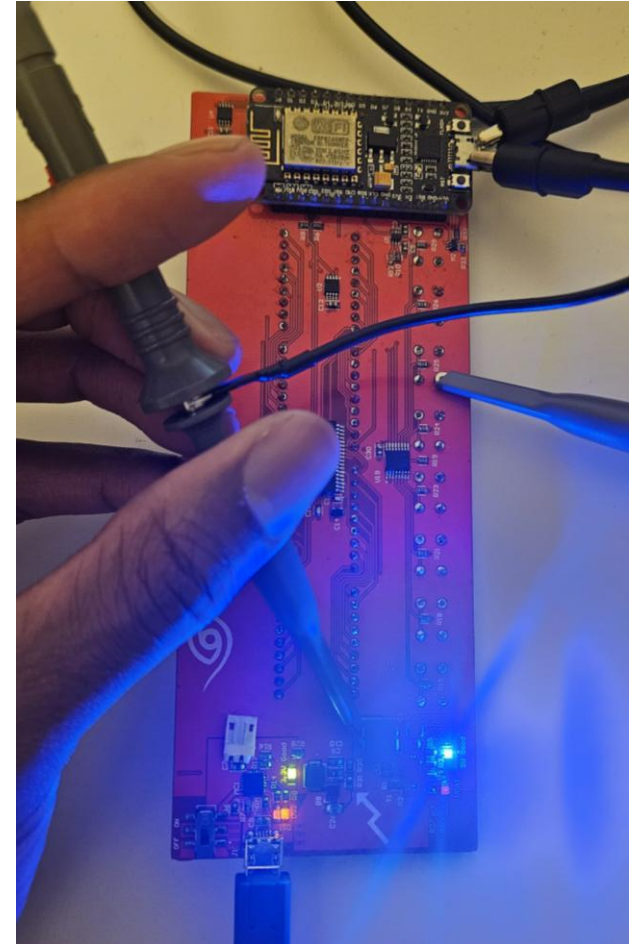


Roadblocks and Debugging

- When I soldered on NeoPixel LEDs, all power good LEDs went off and everything on board instantly shut down 😞

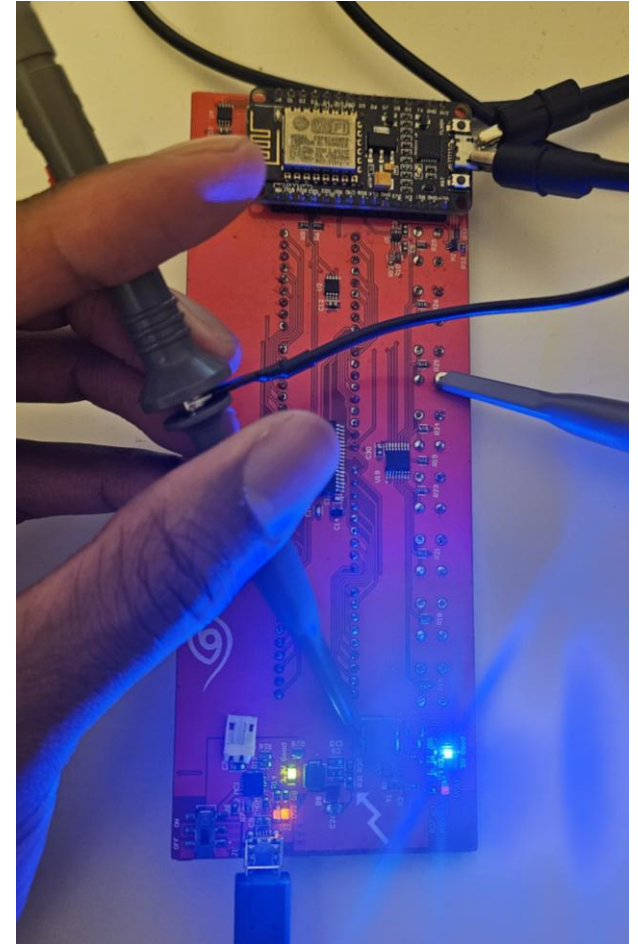
Investigation Steps

- Desoldered the NeoPixels and the board worked again.
- Verified footprint and pinout was correct.
- Verified once more 5V was good.
- Tried with other Neopixels but saw the same behavior.

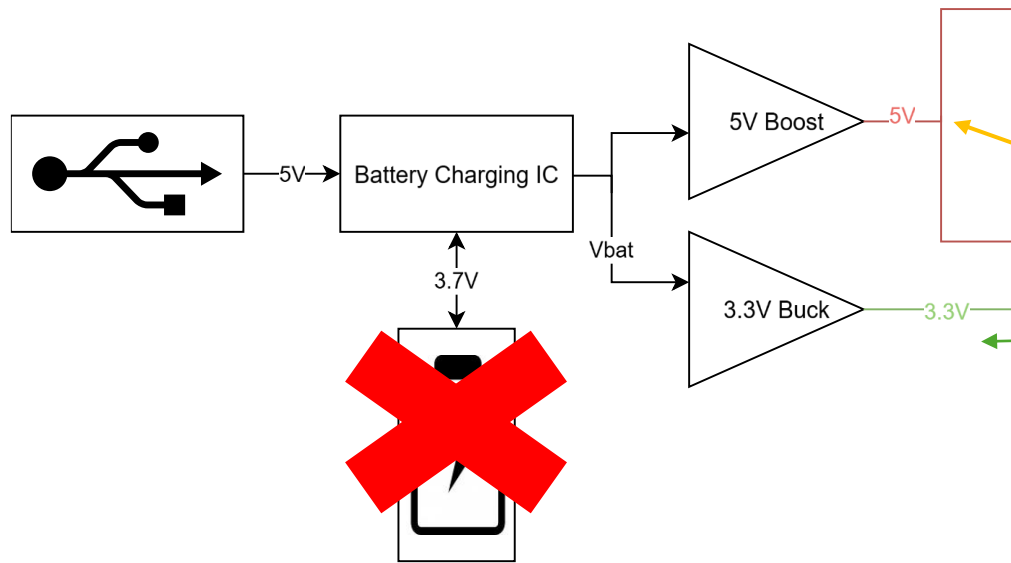


Investigation Steps

- I started attaching loads to the 5V rail and checking the voltage; it was fine.
- However, I noticed when I used a 47.5Ohm power resistor load, the 3.3V LED no longer lit up...
- Time to scope...

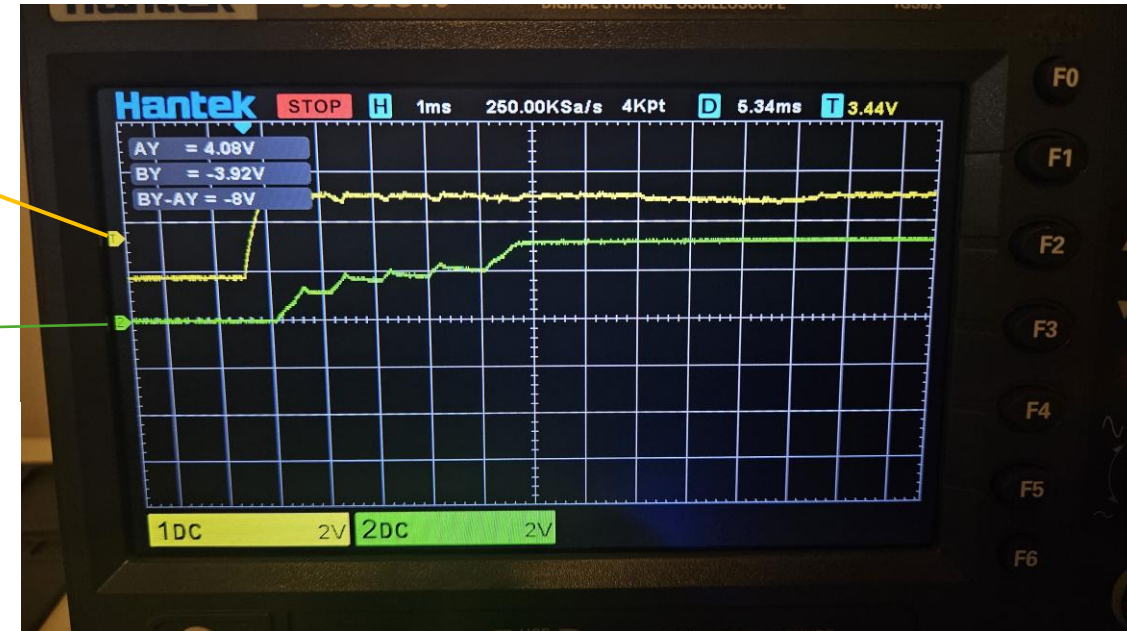
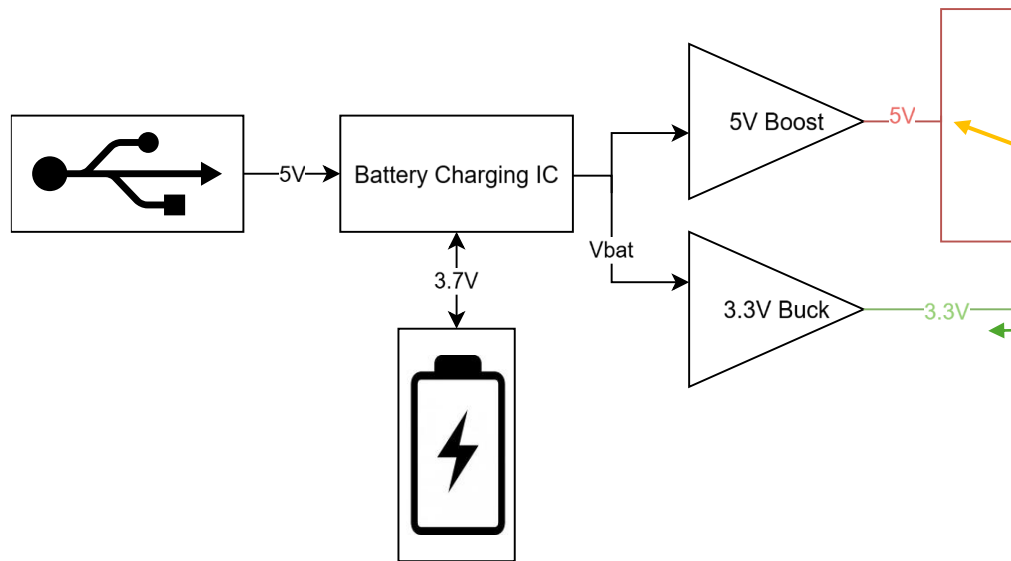


Debugging



Without Battery
3.3V takes **9ms** to ramp up!

Debugging

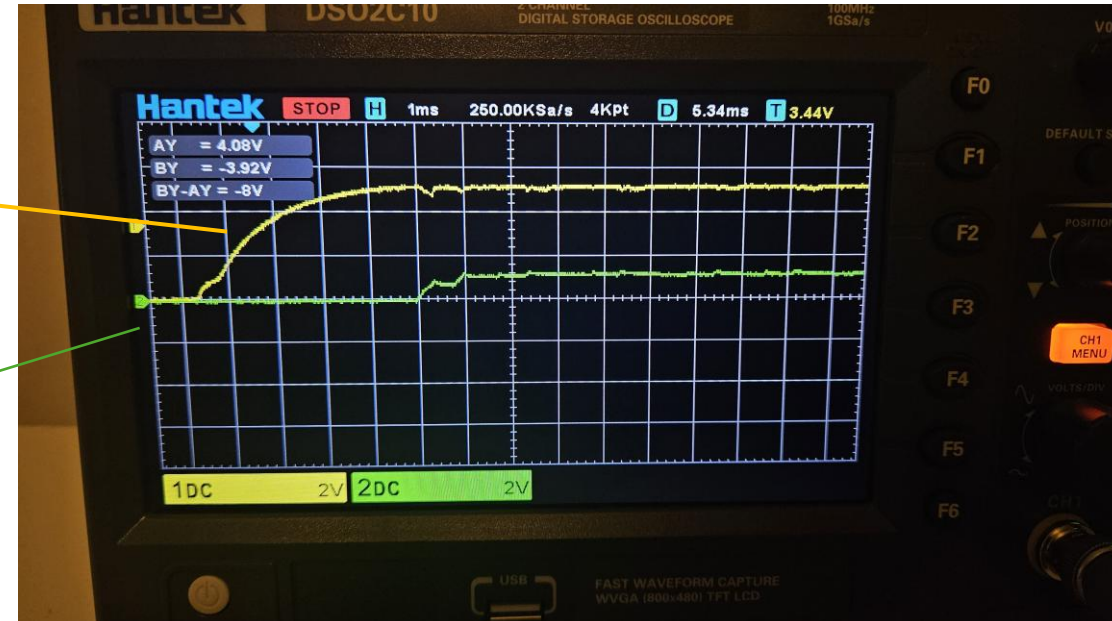
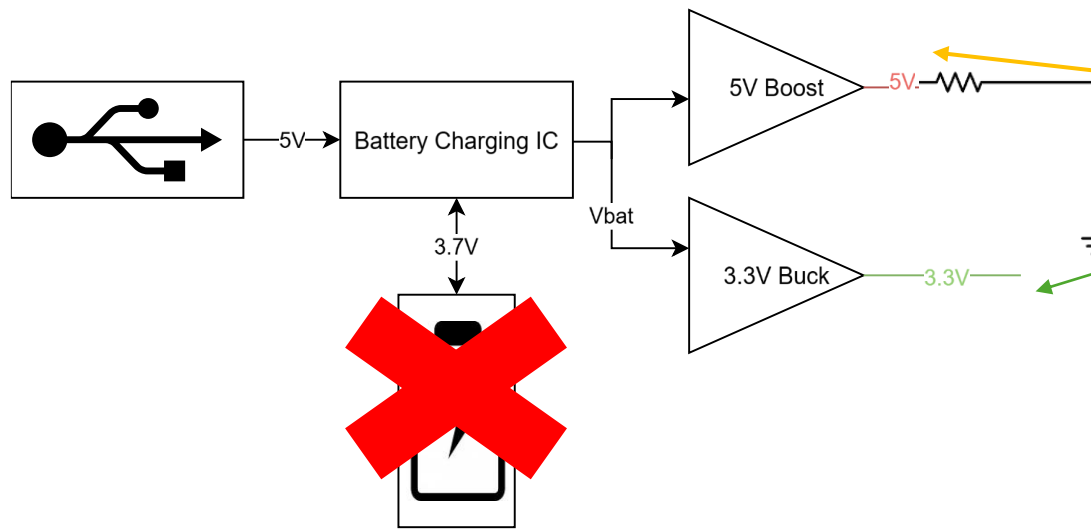


With Battery
3.3V takes **5ms** to ramp up! Much faster!

Debugging

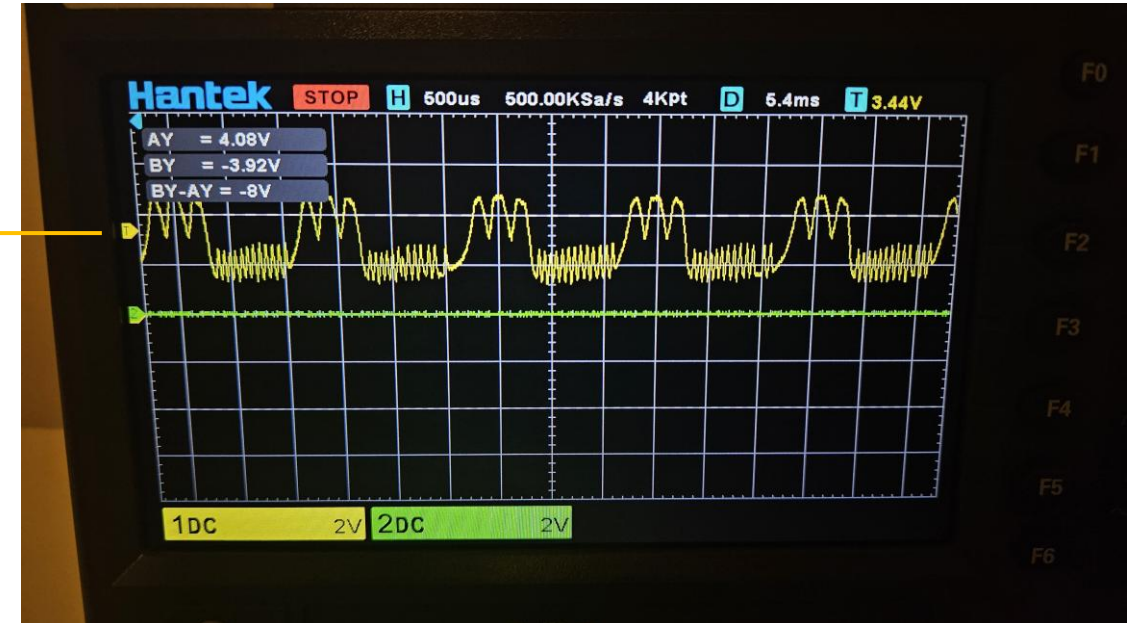
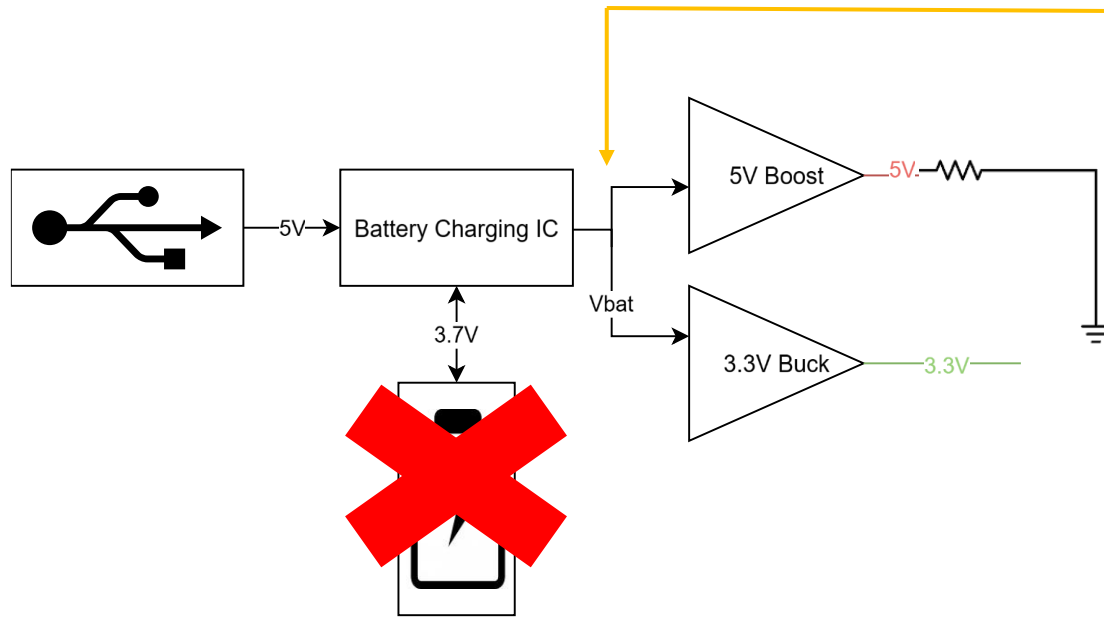
- I wanted to recreate the NeoPixel fault condition, which was with **no battery** and a **load on 5V**

Debugging



With 47.5 Ohm load on 5V
(simulating a NeoPixel), the
3.3V rail never gets to 3.3V!

Debugging



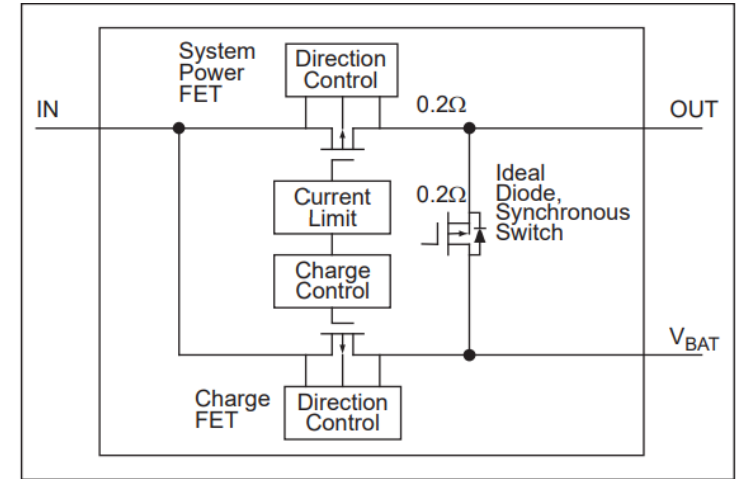
This showed as 2.7V on a multimeter.

Buck converter cannot reduce 2.7V to 3.3V

Debugging

I looked at the datasheet which says that the chip *should* be able to work with or without a battery

However, with not battery, I was seeing the OUT voltage drop with a load



Conclusion

With OUT voltage too low, the buck converter cannot operate any longer, but the boost converter still boosts this lower voltage up to 5V!

Hypothesis:

With a larger load (more current), OUT voltage can drop even more. If it falls below 1.8V, the boost converter can no longer function, shutting everything down.

Workaround

- The ESP8266 and 3.3V worked without the NeoPixels, so I made the tradeoff of leaving them off and continuing with other parts of the project

Note:

I did this investigation after I removed the NeoPixels and haven't resoldered and tested again.

Firmware (1/3)

- Made drivers to handle I2C register reads/writes for GPIO Expander, 14-Segment Controller, Temperature Sensors
- Set the GPIO expander interrupt pin to trigger a read of the button states
- Then made functions to abstract away the lowest level functions

Firmware (2/3)

MAX6955 14-Segmet Controller Code Example

```
void initMAX6955()
{
    //config reg, set normal operation
    writeMAX6955(max6955_reg_configuration, 0x01);

    // ascii decoding for all digits;
    writeMAX6955(max6955_reg_decodeMode, 0x0F);

    // brightness: 0x00 = 1/16 (min on) 2.5 mA/seg;
    // 0x0F = 15/16 (max on) 37.5 mA/segment
    writeMAX6955(max6955_reg_globalIntensity, 0x07);

    // active displays: 0x07 -> all;
    writeMAX6955(max6955_reg_scanLimit, 0x07);

    //digittype (14seg)
    writeMAX6955(max6955_reg_digitType, 0xFF);
}
```

```
void writeMAX6955(byte command, byte data)
{
    Wire.beginTransaction(MAX6955_I2C_ADDR);
    Wire.write(command);
    Wire.write(data);
    Wire.endTransmission();
}

void writeDisplay(String msg)
{
    byte j = 0;
    bool dotLit = false;
    for (byte i=0; i < 8; i++)
    {
        if(i + 0x01 < (byte)msg.length() && msg[i+1] == '.'){
            dotLit = true;
        }

        if (i < msg.length())
            writeMAX6955(0x67-j, (dotLit? 0x80 : 0) | msg[i]);
        else
            writeMAX6955(0x67-j, ' ');

        if(dotLit){
            i++;
        }
        j++;
        dotLit = false;
    }
}
```

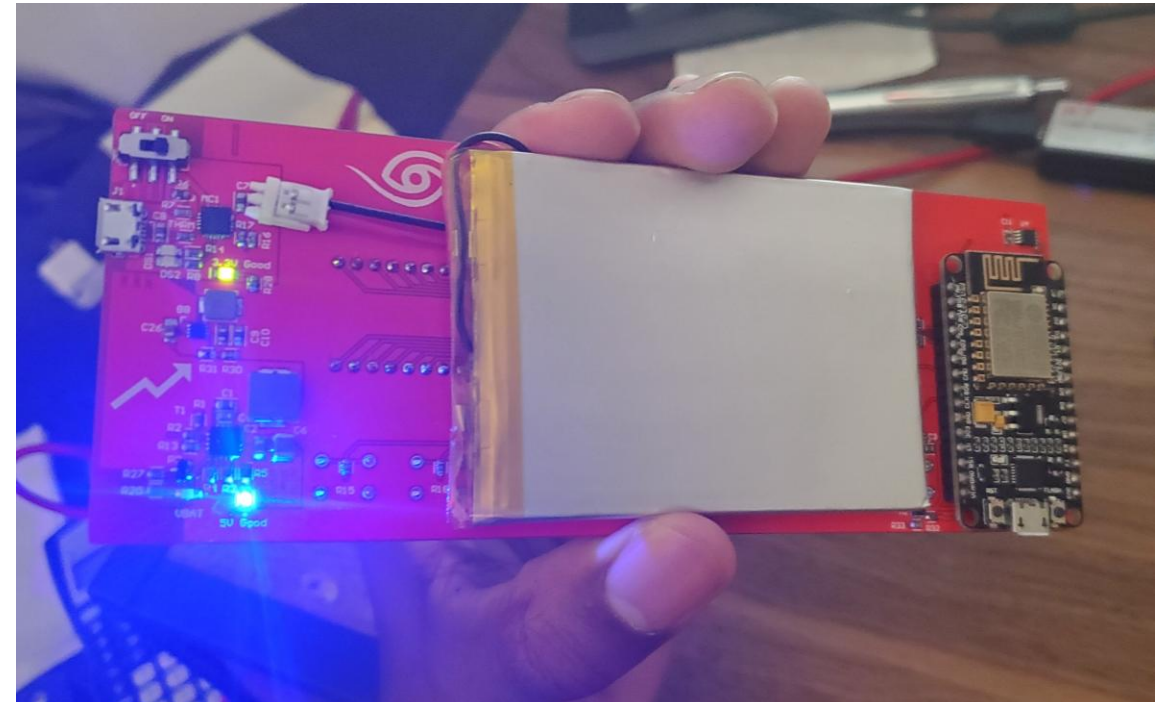
Software (3/3)

- Have state machine for each mode
- Each mode is an entire additional and unique state machine that handles inputs from 'Enter' or Buttons #0-#5, makes API calls, updates the display, and whatever else it needs to do

```
void state_machine(){  
    switch(GLOBAL_STATE){  
        case 0:  
            STATE_Temperature();  
            break;  
        case 1:  
            STATE_Stock();  
            break;  
        case 2:  
            STATE_ChessClock();  
            break;  
        case 3:  
            STATE_Clock();  
            break;  
        default:  
            GLOBAL_STATE = 0;  
            break;  
    }  
}
```

Results and Validation

- Board worked as intended otherwise!
- Able to display on all 14-segments, buttons work, connects to Wi-Fi



Lessons Learned

- 4-layer board
- Don't use a devboard
- Different Charging Chip
- Use buck-boost converter on 3.3V rail
- Power up 3.3V first, then use GPIO to enable 5V boost converter from microcontroller to prevent power sequencing issues



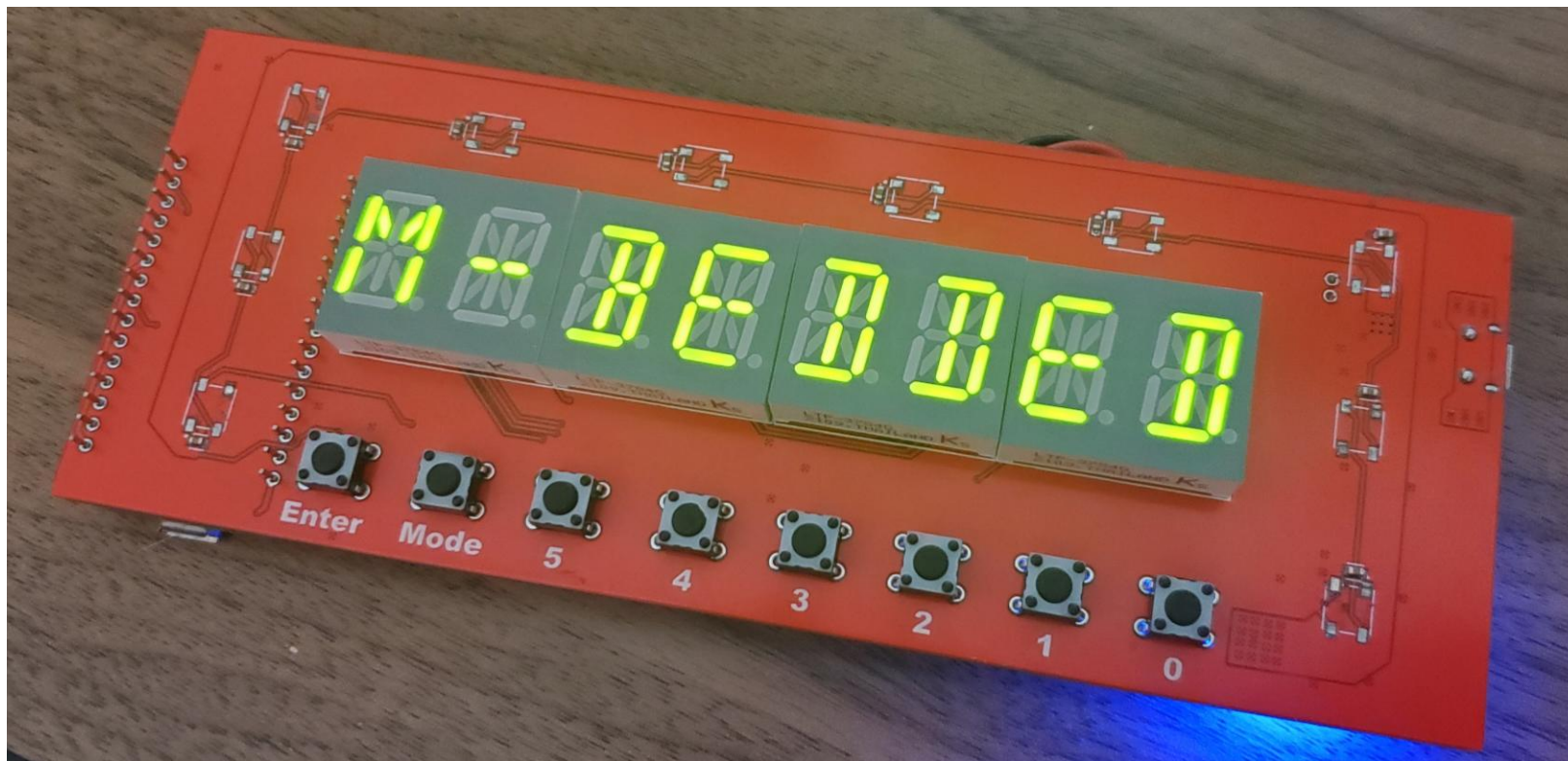
My Engineering Approach

1. Define the ideal outcome
2. Define any requirements & limitations that go along with the ideal outcome
3. Research and tradeoffs
4. Identify any unknowns or risky aspects and test/prototype them ☆
5. Redefine/Shift ideal outcome with data
- 6 . Make it! (and take notes)

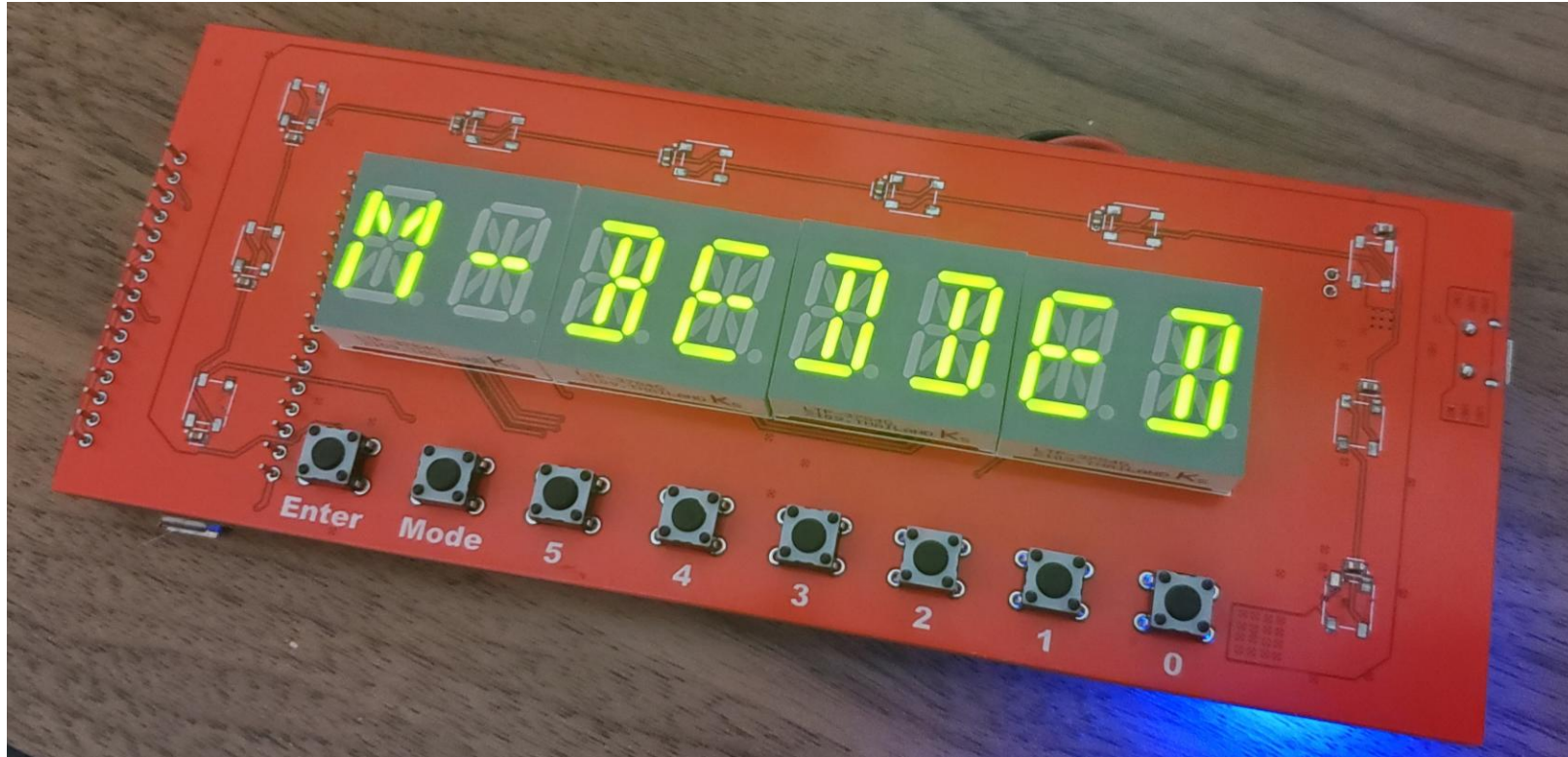
Working in teams

- I like to work on teams where everyone contributes and has a common goal
- I appreciate individual thinking followed by group discussion to work out the best solution
- I value teams who document their knowledge and iterate forward
- I value interfacing between teams (Hardware – Software – Mechanical)

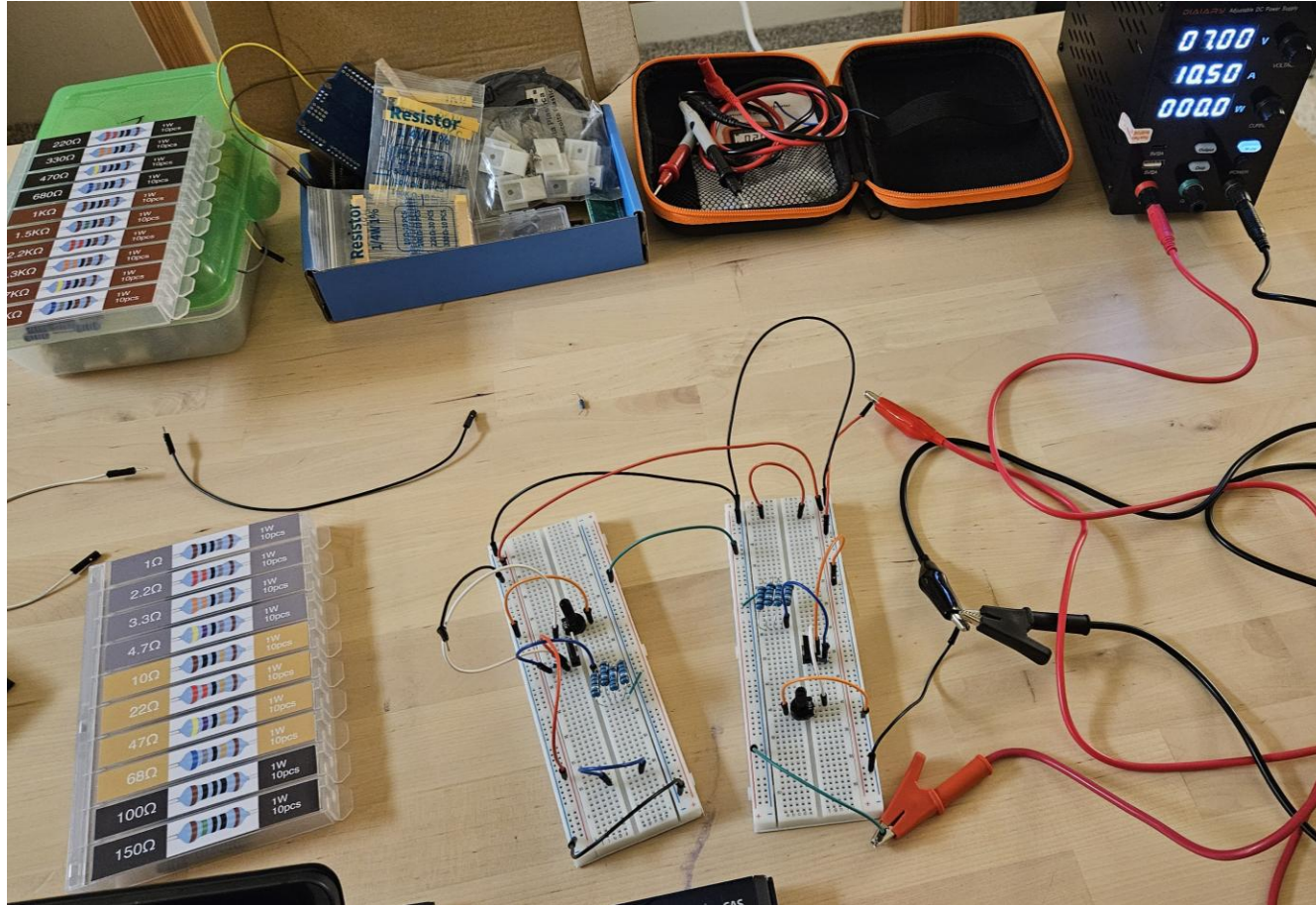
Questions



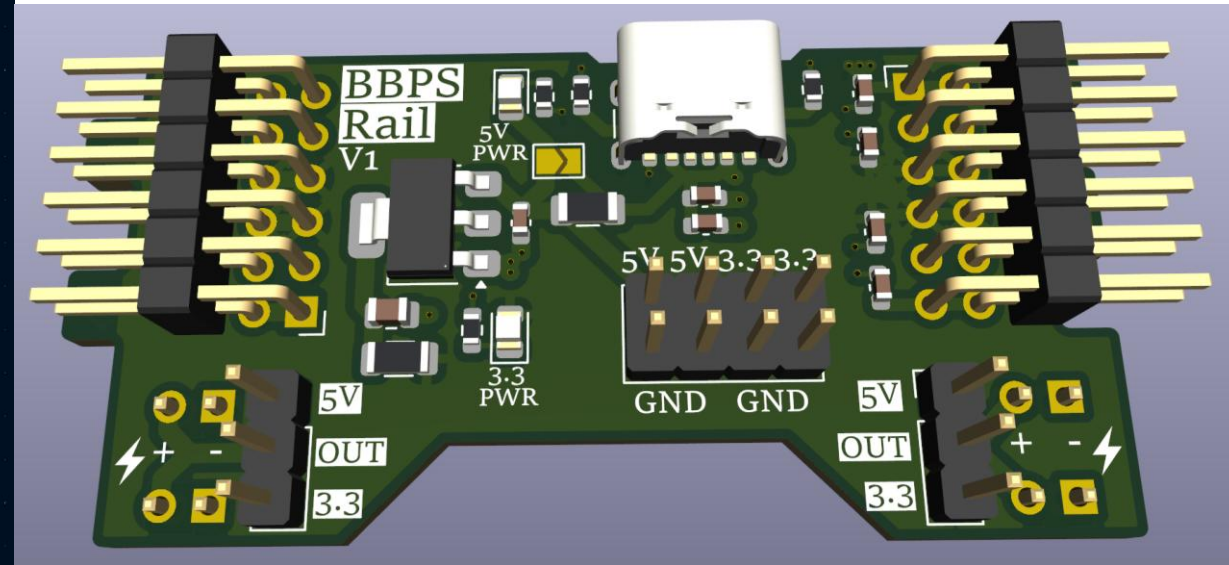
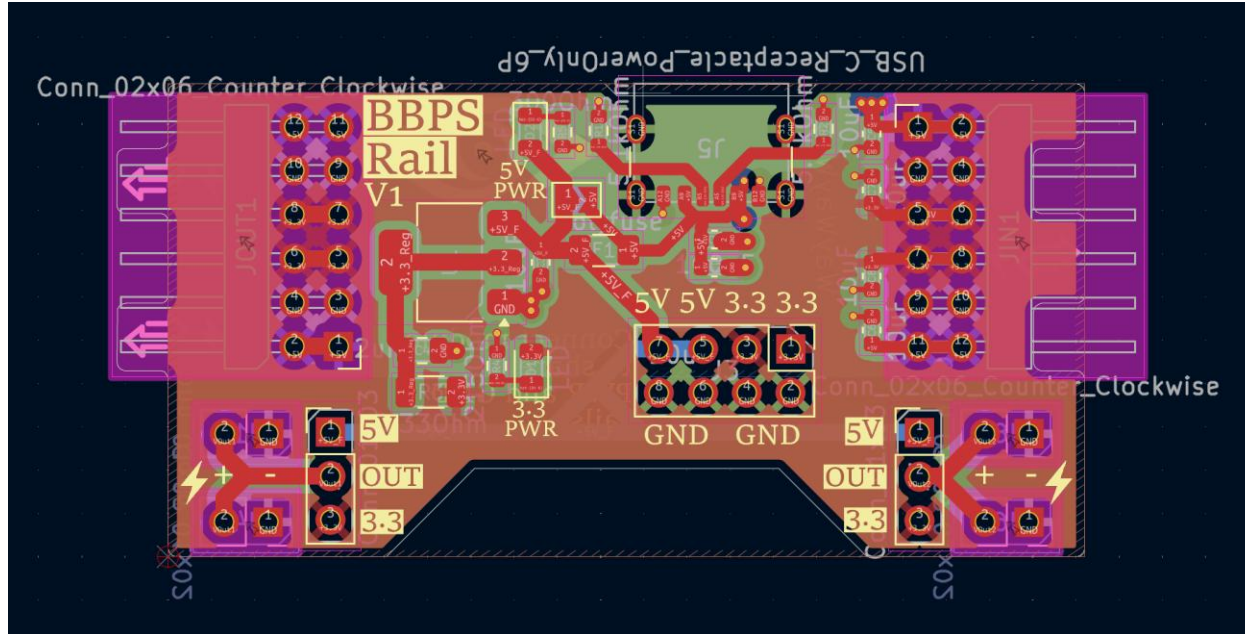
Demo



Extra – Load Sharing Breadboard Power Supply (1/3)



Extra – Load Sharing Breadboard Power Supply (2/3)



Extra – Load Sharing Breadboard Power Supply (3/3)

